

# GROUP A - Frontend Team Guide

M1: Learner UI | M2: Admin UI + i18n (English & Hindi)

## What This Guide Covers

- Complete project overview and your team's exact role
- All topics to learn with free online resource links
- Step-by-step environment & tool installation guide
- Google Antigravity (AGY) usage with example prompts
- Week-by-week 16-week build plan with deliverables
- Integration contracts with other groups
- Testing, common mistakes, and daily workflow

**Primary Development Tool: Google Antigravity (AGY) IDE**

# Group A Frontend Team Guide: AI-Enabled Learning & Competency Gap Platform

## Table of Contents

1. PROJECT OVERVIEW
  2. MEMBER RESPONSIBILITIES
  3. TOPICS TO LEARN
  4. ENVIRONMENT SETUP
  5. ANTIGRAVITY USAGE GUIDE
  6. FOLDER STRUCTURE
  7. COMPONENT GUIDE
  8. API INTEGRATION
  9. HINDI INTERNATIONALIZATION
  10. WEEK-BY-WEEK TASKS
  11. INTEGRATION WITH OTHER GROUPS
  12. TESTING GUIDE
  13. COMMON MISTAKES TO AVOID
  14. DAILY WORKFLOW
  15. RESOURCES & REFERENCE LINKS
- 

## 1. PROJECT OVERVIEW

Welcome to the AI-Enabled Learning & Competency Gap Platform for India's Official Statistical System. This platform is a strategic initiative for SIH 2026. Its core mission is to modernize and scale the competency assessment and training workflows for government officials in the statistical domain, deeply integrating with the iGOT Karmayogi (government training portal) infrastructure.

### The Problem

Currently, government statistical officers face a fragmented ecosystem for identifying what skills they lack (Competency Gaps) and discovering the right courses to bridge those gaps. Needs assessments are largely manual, one-off, and not tied directly to a continuous learning ecosystem.

### The Solution

We are building a holistic platform that:

- Uses AI to evaluate an official's current competency levels through dynamic, adaptive assessments (MCQs, document analysis).
- Highlights gaps against their target role requirements using visual analytics (GapRadar).
- Recommends tailored learning paths integrating iGOT Karmayogi course catalogs.
- Provides analytics and dashboards for departmental heads to view team-wide competency heatmaps.

### Your Role (Group A: Frontend Team)

As Group A, you are the face of this platform. The frontend you build is the interface through which thousands of officials will interact with the system. Your responsibility is to translate complex backend AI models and vast datasets into intuitive, accessible, and fast user interfaces.

This is a massive responsibility because a platform is only as successful as its user adoption. If the Assessment Runner is clunky, users will drop off. If the Analytics dashboards are confusing, managers will ignore them.

---

## 2. MEMBER RESPONSIBILITIES

This team consists of two primary members: M1 and M2. You will work closely together, but to ensure accountability and parallel progress, responsibilities are split as follows.

### M1 Responsibilities: Learner-Facing Interfaces

Your primary focus is the end-user (the official taking the assessment and following the learning path).

- Dashboard with GapRadar Chart: The landing page for a logged-in user. It must visually demonstrate where they stand versus where they need to be.
- Assessment Runner with MCQ Interface: The core test-taking engine. Needs to handle state cleanly (timers, answer selection, navigating previous/next, submission).
- Learning Path Timeline: A vertical or horizontal timeline showing recommended courses, deadlines, and progress.
- Resource Browser: A searchable, filterable directory of iGOT Karmayogi courses and uploaded study materials.
- Practice Quiz Mode: A lighter version of the Assessment Runner for casual learning without high stakes.

### M2 Responsibilities: Admin/Manager Pages & Infrastructure

Your primary focus is on organizational management, data visualization for leaders, and core app infrastructure.

- Competency CRUD & Role Mapping: Interfaces for admins to define what skills are required for a "Junior Statistician" versus a "Senior Director."
  - MCQ Review Panel: An interface for subject matter experts to review AI-generated questions, edit them, and approve them for production.
  - Analytics Dashboards: High-level charts (bar, pie, line) showing system usage, average scores, and completion rates.
  - Team Gap Heatmap: A grid visualization showing which specific skills are lacking across an entire department.
  - i18n Setup (English + Hindi): Establishing the next-intl infrastructure, ensuring all text strings are extracted, and setting up the Hindi font (Noto Sans Devanagari).
- 

## 3. TOPICS TO LEARN

You must understand the following technologies. Do not just skim; deeply comprehend the core concepts.

- HTML5/CSS3 basics: Semantic HTML and modern CSS flexbox/grid. MDN Web Docs
  - JavaScript ES6+: Arrow functions, destructuring, promises, async/await, array methods (map, filter, reduce). javascript.info
  - React 18 fundamentals: Components, JSX, props, state, hooks (useState, useEffect, useMemo, useCallback). react.dev
  - Next.js 14 App Router: Server components vs. Client components, routing conventions, layouts, loading UI, Server Actions. Next.js Docs
  - Tailwind CSS: Utility-first CSS framework. Learn how to compose classes, handle responsive design (md:, lg:), and states (hover:, focus:). Tailwind Docs
  - shadcn/ui components: Reusable components built on Radix UI and Tailwind. You own the code, you can tweak it. shadcn/ui
  - next-intl for i18n: App Router integration for internationalization. next-intl
  - Recharts for data viz: A composable charting library built on React components. Recharts
  - Zustand state management: A small, fast, scalable barebones state-management solution. Use it for global state (e.g., Assessment Runner state). Zustand
  - TanStack Query (React Query): Powerful asynchronous state management. Handles caching, background updates, and stale data. TanStack Query
  - Playwright E2E testing: Reliable end-to-end testing for modern web apps. Playwright
  - Mock Service Worker (MSW): API mocking by intercepting requests on the network level. Crucial for parallel development before Group B finishes APIs. MSW
  - TypeScript basics: Types, interfaces, generics, utility types. TypeScript Handbook
-

## 4. ENVIRONMENT SETUP

We will use a standard environment to ensure "it works on my machine" applies to everyone.

### Step 1: Install Node.js

Ensure you have Node.js 20 LTS installed.

```
node -v
# Should output v20.x.x
```

### Step 2: Install VS Code

Download from [code.visualstudio.com](https://code.visualstudio.com).

Install these extensions:

- Prettier - Code formatter
- ESLint
- Tailwind CSS IntelliSense
- Playwright Test for VSCode

### Step 3: Install Antigravity IDE (Google AGY)

Ensure your Antigravity setup is configured with your standard organizational prompts. This will be your primary AI co-pilot.

### Step 4: Clone and Install

```
# Clone the repository (replace URL with actual)
git clone https://github.com/sih-2026-org/learning-platform.git

# Navigate to the web app
cd learning-platform/apps/web

# Install dependencies
npm install
```

### Step 5: Environment Variables

Create a .env.local file in apps/web:

```
NEXT_PUBLIC_API_URL=http://localhost:3000/api
NEXT_PUBLIC_MSW_ENABLED=true
```

### Step 6: Run Dev Server

```
npm run dev
```

Open <http://localhost:3000> in your browser.

---

## 5. ANTIGRAVITY USAGE GUIDE

Google Antigravity (AGY) is an agentic IDE that will significantly accelerate your development. However, it requires precise steering.

### Bounded Prompts

Do not ask AGY to "build the assessment page." Ask it to "scaffold a React component named AssessmentRunner that

takes questions: `Question[]` as a prop and displays one question at a time with 'Next' and 'Previous' buttons using Tailwind for styling."

## Example Prompts

For M1 (GapRadar):

"AGY, scaffold a GapRadar component using recharts. It should accept a prop data which is an array of objects { subject: string, current: number, target: number }. Plot both current and target on a RadarChart. Use #3b82f6 for current and #10b981 for target. Make it responsive."

For M2 (Role Mapping CRUD):

"AGY, create a Next.js Server Action in actions/roles.ts to create a new Role. It should take a FormData object, extract roleName and description, validate it using Zod, and mock a database insertion. Then, create a client component form using shadcn/ui components to submit to this action."

## Generating Tests

"AGY, read components/AssessmentRunner.tsx. Generate a Playwright test file e2e/assessment.spec.ts that navigates to the assessment page, selects an answer for the first question, clicks 'Next', and verifies the second question is displayed."

## Limitations

- AGY does not know your exact API contract unless you provide the JSON schema.
- AGY might hallucinate Tailwind classes or import paths. Always verify the output.
- AGY should not be used to blindly refactor critical state logic without a human review loop.

---

## 6. FOLDER STRUCTURE

The apps/web directory uses the Next.js App Router paradigm.

```

apps/web/
??? public/           # Static assets (images, fonts)
?   ??? locales/      # Translation files (en.json, hi.json)
?   ??? mockServiceWorker.js# MSW worker script
??? src/
?   ??? app/          # App Router pages and layouts
?   ?   ??? [locale]/  # Dynamic route for i18n (/en or /hi)
?   ?   ?   ??? (auth)/  # Grouped routes for authentication
?   ?   ?   ?   ??? login/page.tsx
?   ?   ?   ?   ??? register/page.tsx
?   ?   ?   ??? (dashboard)/# Grouped routes for logged-in users
?   ?   ?   ?   ??? layout.tsx # Sidebar and Header
?   ?   ?   ?   ??? learner/page.tsx
?   ?   ?   ?   ??? admin/page.tsx
?   ?   ?   ??? layout.tsx # Root layout, injects next-intl provider
?   ??? components/    # React components
?   ?   ??? ui/        # shadcn/ui generic components (Button, Input)
?   ?   ??? learner/    # Learner specific components (GapRadar)
?   ?   ??? admin/      # Admin specific components (Heatmap)
?   ??? lib/           # Utilities
?   ?   ??? utils.ts    # Tailwind cn() utility
?   ?   ??? store.ts    # Zustand global stores
?   ?   ??? api.ts      # Axios/Fetch wrappers
?   ??? hooks/         # Custom React hooks & React Query hooks
?   ?   ??? use-assessment.ts
?   ?   ??? use-competencies.ts
?   ??? mocks/         # MSW definitions
?   ?   ??? browser.ts
?   ?   ??? handlers.ts # Mock API endpoints

```

## 7. COMPONENT GUIDE

### GapRadar (M1)

Purpose: Visualizes user competencies vs requirements.

Props Interface:

```

interface GapData {
  subject: string;
  current: number;
  target: number;
  fullMark: number;
}
interface GapRadarProps {
  data: GapData[];
  title?: string;
}

```

API Endpoint: GET /api/users/{id}/competency-gaps

### AssessmentRunner (M1)

Purpose: The MCQ test engine.

Props Interface:

```
interface Question {
  id: string;
  text: string;
  options: { id: string; label: string }[];
}
interface AssessmentRunnerProps {
  assessmentId: string;
  initialQuestions: Question[];
}
```

State Management: Use Zustand to track currentQuestionIndex, answers: Record<string, string>, and timeRemaining.

### QuizReviewPanel (M2)

Purpose: Admin interface to approve AI-generated questions.

Props Interface:

```
interface GeneratedQuestion extends Question {
  confidenceScore: number;
  sourceMaterialId: string;
}
interface QuizReviewPanelProps {
  batchId: string;
  questions: GeneratedQuestion[];
}
```

API Endpoint: PUT /api/admin/questions/{id}/approve

### LanguageSwitcher (M2)

Purpose: Toggles between English and Hindi.

Implementation: Uses Next.js router to switch the [locale] segment in the URL.

## 8. API INTEGRATION

Until Group B finishes the backend APIs, we use MSW.

### MSW Setup (src/mocks/handlers.ts)

```
import { http, HttpResponse } from 'msw'

export const handlers = [
  http.get('/api/users/:id/competency-gaps', ({ params }) => {
    return HttpResponse.json([
      { subject: 'Data Analysis', current: 60, target: 80, fullMark: 100 },
      { subject: 'Statistical Modeling', current: 40, target: 90, fullMark: 100 },
      { subject: 'Survey Design', current: 75, target: 80, fullMark: 100 },
    ])
  }),
]
```

### React Query Hook Example (src/hooks/use-competencies.ts)

```
import { useQuery } from '@tanstack/react-query';

export function useCompetencyGaps(userId: string) {
  return useQuery({
    queryKey: ['competency-gaps', userId],
    queryFn: async () => {
      const res = await fetch(`/api/users/${userId}/competency-gaps`);
      if (!res.ok) throw new Error('Network response was not ok');
      return res.json();
    },
  });
}
```

## 9. HINDI INTERNATIONALIZATION

The platform must support English (en) and Hindi (hi).

### next-intl Setup

1. Define translations in messages/en.json and messages/hi.json.

```
// messages/hi.json
{
  "Dashboard": {
    "title": "????????",
    "welcome": "????? ?? ?????? ??, {name}"
  }
}
```

2. In your Server Components, use getTranslations.

```
import { getTranslations } from 'next-intl/server';

export default async function DashboardPage() {
  const t = await getTranslations('Dashboard');
  return <h1>{t('title')}</h1>;
}
```

### Font Setup

We use Noto Sans Devanagari for Hindi text. Configure it in app/[locale]/layout.tsx:

```
import { Noto_Sans_Devanagari } from 'next/font/google';

const notoSansDevanagari = Noto_Sans_Devanagari({
  subsets: ['devanagari'],
  weight: ['400', '700'],
  variable: '--font-noto-devanagari',
});
```

Update tailwind.config.ts to include this variable in the font family.

## 10. WEEK-BY-WEEK TASKS

### Week 1-2: Scaffolding and Infrastructure

- M1: Setup Next.js, Tailwind, shadcn/ui. Build the base Layout (Sidebar, Header).
- M2: Setup next-intl for i18n, MSW for mock APIs, configure Zustand and TanStack query providers.

### Week 3-4: Learner Dashboard & Data Viz

- M1: Build GapRadar using Recharts. Build the Learner Dashboard page. Implement the RecommendationCard.
- M2: Build the Admin Dashboard. Implement the TeamGapHeatmap component using grid layouts.

### Week 5-7: Assessment Engine

- M1: Build the core AssessmentRunner. Handle state transitions, timers, and submission logic.
- M2: Build the QuizReviewPanel for admins to review AI questions. Create the API endpoints in MSW.

### Week 8-10: Learning Paths & Resources

- M1: Build the LearningPathTimeline and ResourceBrowser integrating mock data from iGOT.
- M2: Build the Competency CRUD interfaces. Allow mapping roles to specific competencies.

### Week 11-12: Integration with Backend

- M1 & M2: Turn off MSW. Connect TanStack Query hooks to the real Group B APIs. Resolve CORS and payload format issues.

### Week 13-14: E2E Testing & Localization

- M1: Write Playwright tests for Learner flows (Assessment taking, viewing dashboard).
- M2: Finalize Hindi translations. Ensure UI doesn't break with longer Hindi strings. Write Playwright tests for Admin flows.

### Week 15-16: Polish, Performance & Handoff

- M1 & M2: Lighthouse audits (accessibility, performance). Fix memoization issues in React. Finalize documentation.
- 

## 11. INTEGRATION WITH OTHER GROUPS

You depend heavily on Group B (Backend API).

- Day 1 Task: Agree on JSON Schemas for core entities (User, Competency, Question, AssessmentResult). Create a shared repository or document for these swagger definitions.
  - Syncs: Bi-weekly syncs with Group B.
  - Contract Testing: Use your MSW mocks as a contract. If Group B changes a payload shape, they must notify you immediately.
- 

## 12. TESTING GUIDE

Use Playwright for E2E tests.

```
// e2e/assessment.spec.ts
import { test, expect } from '@playwright/test';

test('Learner can complete an assessment', async ({ page }) => {
  await page.goto('/en/learner/assessments/123');

  // Verify first question is visible
  await expect(page.locator('text=What is a P-value?')).toBeVisible();

  // Select an option
  await page.click('text=A statistical measure');

  // Go to next
  await page.click('button:has-text("Next")');

  // Submit
  await page.click('button:has-text("Submit Assessment")');

  // Verify redirect to results
  await expect(page).toHaveURL(/.*\/results/);
  await expect(page.locator('text=Assessment Complete')).toBeVisible();
});
```

Run tests: `npx playwright test`

## 13. COMMON MISTAKES TO AVOID

1. Over-fetching data: Use TanStack Query properly. Don't fetch the same competency data in 5 different components. Fetch high up and pass down, or rely on React Query cache.
2. Mutating State Directly in Zustand: Always use the set function. `state.answers[id] = val` is a bug. Use `set((state) => ({ answers: { ...state.answers, [id]: val } } ))`.
3. Hardcoding Strings: Every user-facing string MUST go through `next-intl t('key')`. Do not hardcode English text.
4. Ignoring Accessibility (a11y): Form inputs must have labels. Buttons must have focus states. Use semantic HTML. `shadcn/ui` handles a lot of this, don't break it.
5. Large Client Components: In App Router, keep Client Components ("use client") as small and deep in the tree as possible. Layouts and data fetching should largely be Server Components.

## 14. DAILY WORKFLOW

1. Pull Latest: `git pull origin main`
2. Branch Naming: `feature/M1-gap-radar` or `fix/M2-hindi-font-issue`
3. Coding: Use Antigravity IDE. Scaffold, iterate, test.
4. Commits: Use conventional commits: `feat: add gap radar chart`, `fix: resolve hydration error on dashboard`.
5. PR Process: Never merge your own PR. M1 reviews M2, M2 reviews M1. Check for typescript errors, console logs, and verify tests pass.

## 15. RESOURCES & REFERENCE LINKS

- Figma Designs: [\[Link to Figma\]](#)
- Backend API Swagger/OpenAPI: [\[Link to Swagger\]](#)
- iGOT Karmayogi Documentation: [\[Link to iGOT\]](#)
- SIH 2026 Problem Statement Portal: [\[Link to SIH\]](#)

Team Communication Channel: Slack #sih-group-a-frontend

---

Document Version: 1.0.0

Last Updated: 2026-09-02